Implementation and Performance Analysis of Parallel FFT Algorithms: Cooley-Tukey, Bailey

Koleti Yashwanth Chowdary co23btech11009
Sai Deepak Sana cs23btech11055

April 30, 2026

Abstract

This report details the implementation and performance analysis of the Fast Fourier Transform (FFT). Specifically, we implemented the Cooley-Tukey algorithm in both recursive and iterative forms, as well as Bailey's algorithm. For each algorithmic approach, both sequential and parallel versions were developed. The performance of these implementations was benchmarked across various signal sizes ($N = 2^1$ to $N = 2^{25}$) and thread counts (1 to 64) to evaluate their scalability, efficiency, and speedup characteristics relative to each other and against the highly optimized FFTW library.

1 Introduction

The Fast Fourier Transform (FFT) is a fundamental algorithm in digital signal processing used to compute the Discrete Fourier Transform (DFT) of a sequence. While the naive DFT has a computational complexity of $O(N^2)$, the FFT reduces this to $O(N \log N)$ by taking advantage of symmetries in the complex roots of unity. This project focuses on implementing and comparing sequential and parallel versions of the Cooley-Tukey (recursive and iterative) and Bailey's FFT algorithms, and contextualizing their performance against a FFTW baseline.

2 Methodology

2.1 Sequential Implementations

We implemented three variations of sequential FFT algorithms:

- **Recursive Cooley-Tukey**: The classic divide-and-conquer approach, recursively splitting the DFT into even and odd indices. While mathematically intuitive, it is prone to high function call overhead for large N.
- **Iterative Cooley-Tukey**: This approach avoids recursion by first applying a bit-reversal permutation to the input array, followed by an iterative bottom-up computation of the butterfly operations.
- **Sequential Bailey's**: Bailey's algorithm logically reformulates the 1D FFT of size N into a 2D FFT of size $N_1 \times N_2$. It requires transposing the data matrices, which adds overhead sequentially but lays the groundwork for efficient parallelization.

2.2 Parallel Implementations

To leverage multi-core processors, parallel versions of the algorithms were developed:

- **Recursive Parallel**: Implemented using `std::thread`. At each recursive step, if the signal size is large enough (e.g., $N > 2048$) and threads are available, the computation for the even and odd halves is delegated to separate threads.
- **Iterative Parallel**: Implemented using `pthreads` and barrier synchronization. The bit-reversal is performed sequentially, but the butterfly stages are parallelized by dividing operations among available threads. A `pthread_barrier_t` ensures all threads complete a stage before moving to the next.
- **Bailey's Parallel**: This implementation utilizes `std::thread` to perform parallel matrix transpositions and parallel 1D FFT computations on the rows of the conceptually 2D matrix. This minimizes synchronization overhead compared to the fine-grained barriers in the iterative approach.

3 Experimental Setup

The algorithms were benchmarked on a MacBook Air M4(10-core CPU). Each implementation was executed 20 times, and the average execution time was recorded to ensure consistency and reduce measurement noise.. Tested signal sizes ranged from $N = 2^1$ to $N = 2^{25}$ $(33,554,432)$. The parallel implementations were tested across varying thread counts: 1, 2, 4, 8, 16, 32, and 64 threads. The "Fastest Fourier Transform in the West" (FFTW) library was benchmarked in parallel execution mode to serve as an upper-bound target for absolute performance.

4 Results and Analysis

4.1 Sequential Performance

Figure 1 illustrates the execution time of the three sequential implementations across all signal sizes.

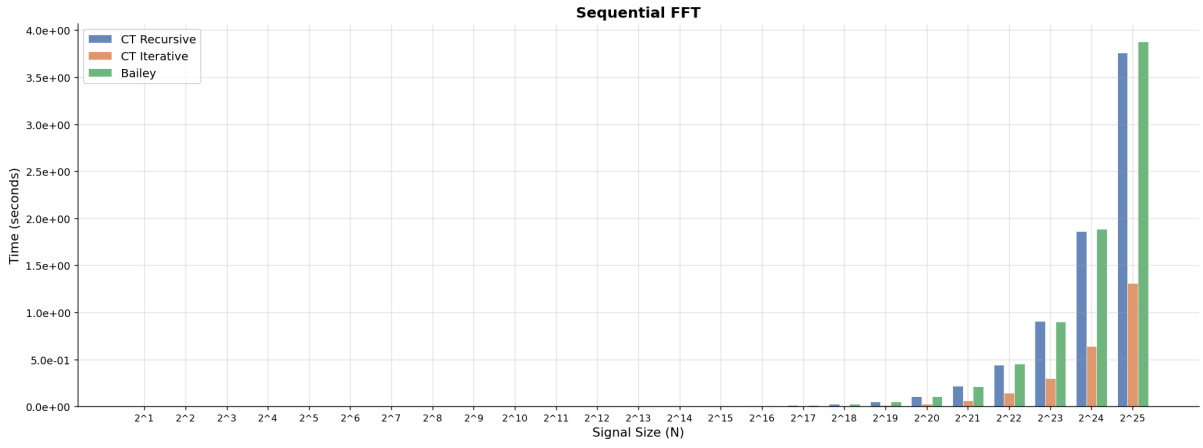


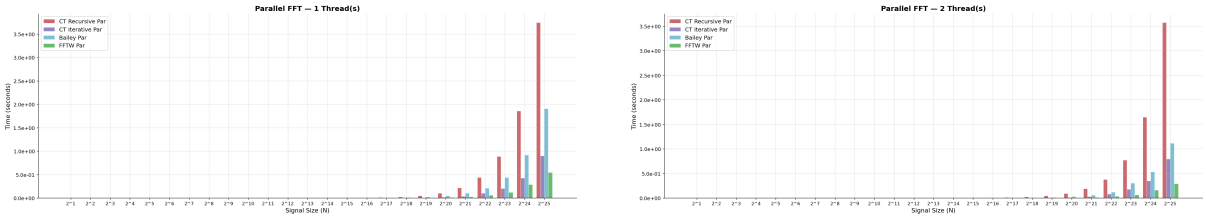
Figure 1: Sequential FFT Performance (CT Recursive, CT Iterative, Bailey's)

Based on the gathered data, the Iterative Cooley-Tukey algorithm is the clear winner in a single-threaded context. At the maximum tested signal size ($N = 2^{25}$), the Sequential Iterative implementation completes in approximately 1.31 seconds, while both the Recursive and Bailey's implementations take ~ 3.76 seconds and ~ 3.88 seconds, respectively. The Iterative algorithm

benefits heavily from the initial bit-reversal sorting. Bailey's algorithm suffers from the memory access overhead of sequentially transposing large arrays.

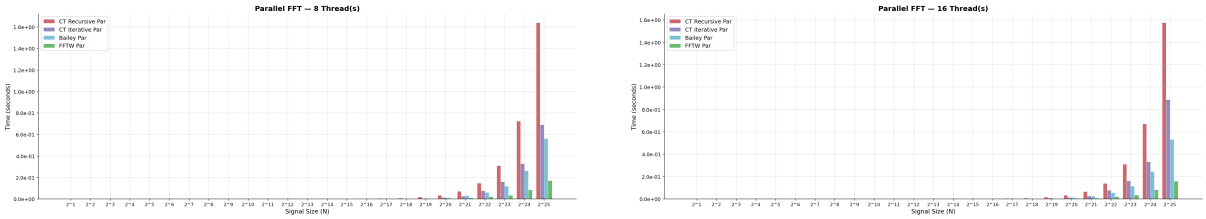
4.2 Parallel Performance and Scaling

The parallel implementations were benchmarked against an optimized FFTW baseline. Figures 2 to 4 depict the execution times across different thread counts.



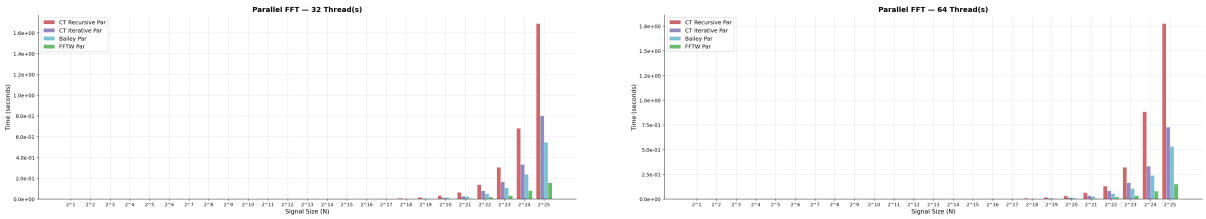
(a) 1 Thread (b) 2 Threads

Figure 2: Parallel FFT Execution Times (1 and 2 Threads)



(a) 8 Threads (b) 16 Threads

Figure 3: Parallel FFT Execution Times (8 and 16 Threads)



(a) 32 Threads (b) 64 Threads

Figure 4: Parallel FFT Execution Times (32 and 64 Threads)

4.2.1 Recursive Parallel Scaling

Table 1 summarizes the speedup achieved by the recursive parallel FFT relative to its sequential version.

Table 1: Speedup of Recursive Parallel FFT vs. Sequential Recursive FFT

Signal Size (N)	2 Threads	4 Threads	8 Threads	16 Threads	32 Threads	64 Threads
2^{20}	1.16x	2.52x	2.98x	3.26x	3.30x	3.39x
2^{21}	1.15x	2.49x	2.98x	3.31x	3.33x	3.26x
2^{22}	1.18x	2.39x	3.01x	3.18x	3.20x	3.34x
2^{23}	1.16x	2.18x	2.91x	2.91x	2.96x	2.80x
2^{24}	1.13x	2.12x	2.57x	2.78x	2.74x	2.10x
2^{25}	1.05x	1.45x	2.29x	2.38x	2.22x	2.11x

The recursive parallel implementation scales poorly. While it achieves a moderate speedup peaking around 3.3x at 16-32 threads, adding more threads yields diminishing returns and eventually performance degradation. The overhead of dynamically spawning and joining threads in a recursive tree heavily outweighs the computational gains, leading to severe thread contention.

4.2.2 Iterative Parallel Scaling

Table 2 summarizes the speedup of the iterative parallel FFT relative to the sequential iterative FFT.

Table 2: Speedup of Iterative Parallel FFT vs. Sequential Iterative FFT

Signal Size (N)	2 Threads	4 Threads	8 Threads	16 Threads	32 Threads	64 Threads
2^{20}	2.09x	2.16x	2.22x	2.21x	2.22x	2.05x
2^{21}	2.11x	2.35x	2.41x	2.40x	2.21x	1.82x
2^{22}	1.79x	1.78x	1.87x	1.77x	1.77x	1.70x
2^{23}	1.70x	1.79x	1.86x	1.86x	1.83x	1.83x
2^{24}	1.83x	1.86x	1.94x	1.92x	1.92x	1.92x
2^{25}	1.65x	1.73x	1.90x	1.48x	1.63x	1.80x

The iterative parallel implementation utilizing `pthread_barrier_t` synchronization exhibits extremely rigid scalability limits. The strict barrier required after every butterfly stage introduces massive wait-time overhead. Even though threads handle isolated indices during a stage, forcing them to sync continuously suffocates the overall scalability.

4.2.3 Bailey's Parallel Scaling

Table 3 details the speedup of the Bailey parallel FFT compared to the sequential Bailey implementation.

Table 3: Speedup of Bailey Parallel FFT vs. Sequential Bailey FFT

Signal Size (N)	2 Threads	4 Threads	8 Threads	16 Threads	32 Threads	64 Threads
2^{20}	3.52x	5.58x	7.96x	7.95x	8.28x	8.71x
2^{21}	3.59x	5.51x	6.78x	7.88x	8.60x	8.26x
2^{22}	3.68x	5.91x	7.47x	8.01x	8.61x	8.55x
2^{23}	2.96x	5.49x	7.61x	7.99x	8.32x	8.51x
2^{24}	3.52x	5.86x	7.21x	7.69x	7.93x	7.93x
2^{25}	3.47x	5.54x	6.91x	7.28x	7.11x	7.29x

Bailey's algorithm demonstrates the most robust scalability of the custom implementations. It achieves strong speedups up to 8 threads and continues to scale gradually, plateauing around 8.7x at 64 threads. This coarse-grained parallelism completely bypasses the fine-grained synchronization bottlenecks seen in the iterative approach.

4.3 Algorithm Comparison vs. FFTW Baseline

We compared the absolute execution times of all implementations against the highly-optimized FFTW library for the maximum signal size ($N = 2^{25}$). The following tables illustrate the raw execution times and the performance gap between our implementations and FFTW across all tested thread counts.

Table 4: Algorithm Performance Comparison at $N = 2^{25}$ with 1 Thread

Algorithm	**Execution Time (Seconds)**	**Ratio vs. FFTW**
FFTW (Baseline)	0.546 s	1.00x
Iterative Parallel	0.901 s	1.65x Slower
Bailey's Parallel	1.903 s	3.49x Slower
Recursive Parallel	3.742 s	6.86x Slower

Table 5: Algorithm Performance Comparison at $N = 2^{25}$ with 2 Threads

Algorithm	**Execution Time (Seconds)**	**Ratio vs. FFTW**
FFTW (Baseline)	0.292 s	1.00x
Iterative Parallel	0.795 s	2.72x Slower
Bailey's Parallel	1.117 s	3.82x Slower
Recursive Parallel	3.574 s	12.24x Slower

Table 6: Algorithm Performance Comparison at $N = 2^{25}$ with 4 Threads

Algorithm	**Execution Time (Seconds)**	**Ratio vs. FFTW**
FFTW (Baseline)	0.202 s	1.00x
Bailey's Parallel	0.700 s	3.47x Slower
Iterative Parallel	0.757 s	3.75x Slower
Recursive Parallel	2.586 s	12.80x Slower

Table 7: Algorithm Performance Comparison at $N = 2^{25}$ with 8 Threads

Algorithm	**Execution Time (Seconds)**	**Ratio vs. FFTW**
FFTW (Baseline)	0.172 s	1.00x
Bailey's Parallel	0.561 s	3.27x Slower
Iterative Parallel	0.690 s	4.02x Slower
Recursive Parallel	1.639 s	9.56x Slower

Table 8: Algorithm Performance Comparison at $N = 2^{25}$ with 16 Threads

Algorithm	**Execution Time (Seconds)**	**Ratio vs. FFTW**
FFTW (Baseline)	0.158 s	1.00x
Bailey's Parallel	0.533 s	3.37x Slower
Iterative Parallel	0.887 s	5.62x Slower
Recursive Parallel	1.575 s	9.98x Slower

Table 9: Algorithm Performance Comparison at $N = 2^{25}$ with 32 Threads

Algorithm	**Execution Time (Seconds)**	**Ratio vs. FFTW**
FFTW (Baseline)	0.158 s	1.00x
Bailey's Parallel	0.545 s	3.44x Slower
Iterative Parallel	0.802 s	5.07x Slower
Recursive Parallel	1.688 s	10.66x Slower

Table 10: Algorithm Performance Comparison at $N = 2^{25}$ with 64 Threads

Algorithm	**Execution Time (Seconds)**	**Ratio vs. FFTW**
FFTW (Baseline)	0.152 s	1.00x
Bailey's Parallel	0.532 s	3.49x Slower
Iterative Parallel	0.729 s	4.78x Slower
Recursive Parallel	1.776 s	11.66x Slower

FFTW's absolute execution time is unmatched. For $N = 2^{25}$, FFTW finishes the computation in just ~ 0.15 seconds on 64 threads. However, Bailey's implementation proves its structural superiority over the other custom approaches, trailing FFTW by only a factor of ~ 3.5x at max capacity, whereas the recursive parallel approach is nearly 12x slower.

5 Conclusion

This project successfully implemented and evaluated sequential and parallel variants of the Cooley-Tukey and Bailey's FFT algorithms. The results yield several key insights:

1. For **sequential execution**, the iterative Cooley-Tukey method is vastly superior due to optimal cache usage, bit-reversal sorting, and zero recursion overhead.

2. **Recursive parallelism** suffers heavily from the prohibitive overhead of managing thread lifecycles in a deep call stack, making it an anti-pattern for large-scale FFTs.

3. **Iterative Cooley–Tukey FFT** shows superior performance over the recursive variant due to its elimination of function call overhead and its more efficient. Its structured, bottom-up computation enables better utilization of modern hardware, whereas the recursive approach suffers from overheads and less predictable data access despite having the same asymptotic complexity.

4. **Data reformulation algorithms**, such as Bailey's method, provide the best parallel scaling. Grouping computations into large, independent chunks allows multi-core systems to operate with minimal synchronization, achieving the highest overall parallel performance.

5. **FFTW** highlights the impact of hardware-specific tuning and SIMD optimizations, achieving extremely high performance that is ultimately constrained by memory bandwidth rather than thread overhead.